J1 Mobile App SQLite Database Plan

Final version with minimal emergency user table

This plan defines the local SQLite database for the J1 mobile app. The app is offline-first: data is stored locally first, then synced later through MQTT/Kafka to the main backend. The local database is not a full copy of the backend database. It only stores the data needed for mobile use, offline reliability, and future synchronization.

1. Design Principles

- Fast emergency usage: users should enter only essential details.
- Offline-first: every report is saved locally before network sync.
- Main DB compatibility: report payloads should match the Prisma IncomingReport structure where possible.
- Mobile simplicity: avoid copying all backend tables and relations into the app.
- Sync safety: failed sync attempts remain in the local sync queue until successful.

2. SQLite Tables Overview

Table	Purpose
user	Stores minimal current user information for emergency reporting.
incoming_reports	Stores SOS/disaster reports created from the mobile app.
sync_queue	Stores pending offline sync operations.
metadata	Stores local app state such as online status and last sync time.
alerts	Caches public alerts received from backend.
confirmed_incidents	Caches active incidents from J3/main system.
resources	Optional cache for rescue resources if needed in the mobile app.

3. User Table - Minimal Emergency Design

Because this is an emergency app, user registration should be quick. The mobile user table should store only the basic details required to identify and contact the reporter. Full user management can be handled later by the backend if needed.

Field	Type	Note
id	TEXT	Local UUID. Needed for local tracking and report ownership.
name	TEXT	Reporter name. Required for emergency follow-up.
telephone	TEXT	Reporter contact number. Required.
device_id	TEXT	Optional. Helps identify the device and reduce duplicates.
created_at	TEXT	UTC timestamp when user record is created.

Recommended user table

```
user(id, name, telephone, device_id, created_at)
```

4. Incoming Reports Table

This table stores the actual emergency reports created in J1. It should be close to the main Prisma IncomingReport model so that later Kafka/main DB integration becomes easier.

- id - local report ID

- source - usually J1_SOS_APP
- disaster_type - FLOOD, LANDSLIDE, OTHER
- district - user selected or detected district
- latitude, longitude - GPS coordinates
- description - incident/help request details
- contact - reporter telephone number from user table
- media_urls - JSON string list of local/remote media references
- verification_status - PENDING_REVIEW initially
- sos_id - local SOS/report ID for backend mapping
- device_id - optional mobile device identifier
- created_at - UTC timestamp
- synced_status - LOCAL_ONLY, QUEUED, SUBMITTED, FAILED

5. Sync Queue Table

The sync_queue table records what must be sent to the backend. It separates local data storage from network transport. If the app closes or network fails, queued items remain safe.

```
sync_queue(queue_id, entity_type, entity_id, operation, payload, status, retry_count, created_at, synced_at)
```

6. Metadata Table

The metadata table stores small key-value state used by the sync engine and UI. Examples: is_online, queue_count, last_sync_time, last_error.

```
metadata(key, value, updated_at)
```

7. Alerts, Incidents, and Resources

These tables are optional caches for data received from the backend/J3 dashboard. They are useful for showing public alerts, active incidents, and available resources in the mobile app.

Table	Fields
alerts	id, type, severity, title, description, district, is_public, is_active, source, created_at, expires_at
confirmed_incidents	id, title, disaster_type, district, severity, status, latitude, longitude, description, affected_people, created_at, updated_at
resources	id, type, name, district, status, latitude, longitude, capacity, current_load, incident_id, last_updated

8. Report Submission Flow

- User enters name and telephone once, then continues quickly.
- User submits an SOS/report form.
- The app saves the report in incoming_reports.
- The app adds the same Prisma-compatible payload to sync_queue.
- When online, SyncService sends queued payloads to MQTT/Kafka.
- After successful delivery, sync_queue status becomes SUBMITTED and report synced_status is updated.

9. Suggested Implementation Order

- Create or update user table with minimal emergency fields.
- Create incoming_reports table and map report form fields to it.
- Create sync_queue and save every outgoing report there.
- Add metadata for network/sync state.
- Add alerts and confirmed_incidents after basic reporting works.

10. Key Decision

The mobile app should not force full user registration. For emergency use, the local user table should keep only id, name, telephone, optional device_id, and created_at. The backend can later enrich or map this into a full user model if required.